SCREENS & DATABASE

Shopify Store Migration App

How every screen looks, how a migration works from start to finish, and the database that stores it — the 12 tables of the **migration_tool** database and how they connect.

Version	1.0
Date	6 October 2026
Database	MySQL · `migration_tool` · 12 tables
Screens	Setup · Dashboard · Migrations (5-step wizard, progress, results) · Plans · Help & support

Contents

1. How the app works
2. Setup screen (first open)
3. Dashboard
4. Migrations list
5. New migration — 5-step wizard
6. Progress screen
7. Results screen
8. Plans and Help & support
9. Database overview and relationship diagram
10. Table by table
11. Relationships and delete rules
12. Example: what each step writes to the database

1. How the app works

The merchant installs the app on their **new Shopify store**, connects their **old store** (WooCommerce, WordPress, …) with its API keys, chooses what to bring across and starts. The data is moved by a background job on the server, so the merchant can close the browser. When it is done they see what moved, what failed and why.

Install Add app from Shopify → **Connect** URL + API keys → **Select** Choose data → **Begin** Demo or full → **Migrate** Background job → **Results** Report, retry

App menu

Menu	What the merchant does there
Dashboard	Sees cards: how many migrations, how many records moved, what is running, recent migrations.
Migrations	Starts a new migration — picks the platform, enters the keys, selects what to migrate, begins — and follows its progress and results.
Plans	Chooses a plan; paid plans are billed through Shopify.
Help & support	Installation steps, how to create API keys, FAQ.

Order the data moves in

Later data links to earlier data, so steps always run in this order:

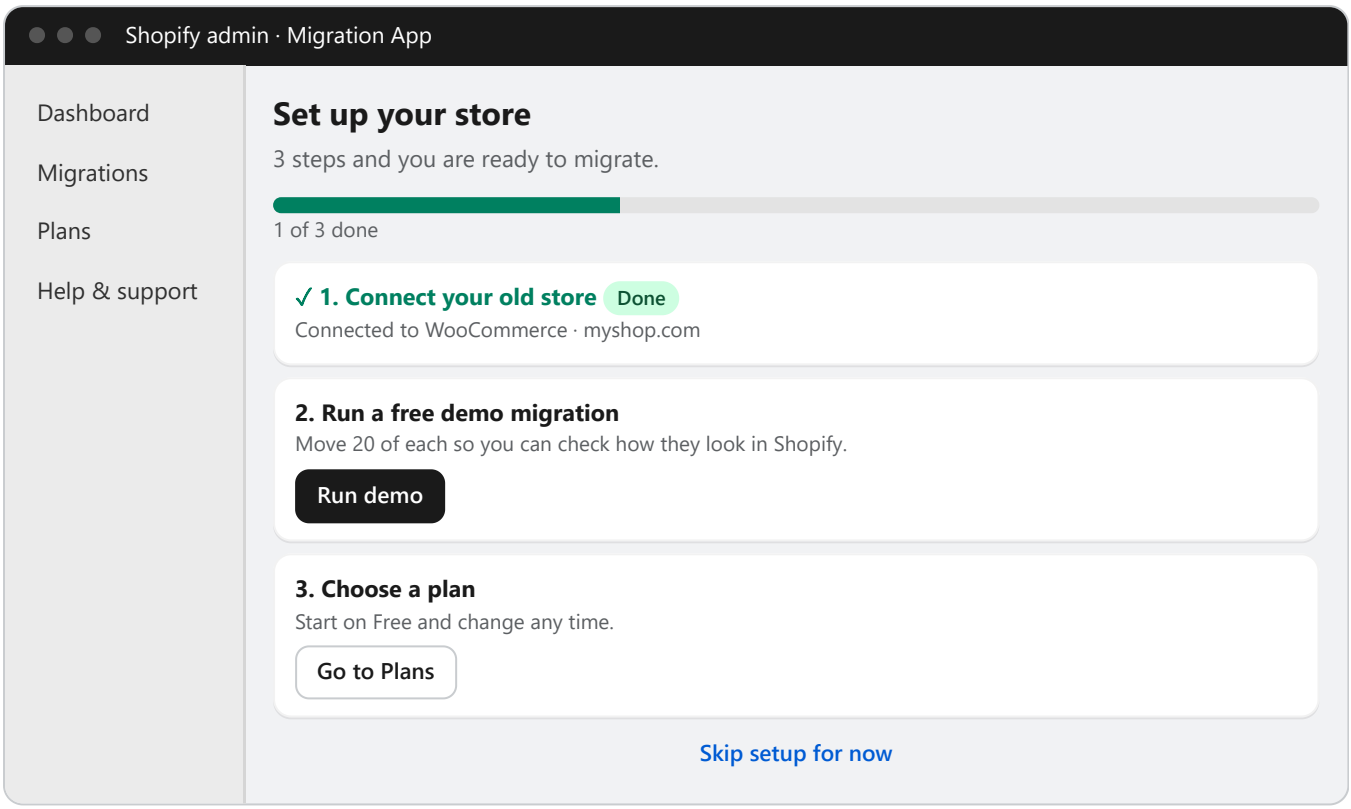
1 Collections → **2** Products → **3** Customers → **4** Orders → **5** Coupons → **6** Pages & posts → **7** Redirects

What the background job does, for every record

1. Reads a page of records (e.g. 50 products) from the old store.
2. Checks `id_map` — if the record was already migrated, it is skipped. No duplicates.
3. Converts it to Shopify's format, swapping old IDs for new Shopify IDs found in `id_map` (so an order points at the right product and customer).
4. Creates it in Shopify.
5. Success → saves the old ID ↔ new ID in `id_map`. Failure → saves the reason in `migration_errors`. One bad record never stops the migration.
6. Saves its position (`source_cursor`) and counters in `migration_steps` after every page — so after a pause or a server restart it continues where it stopped.

2. Setup screen (first open)

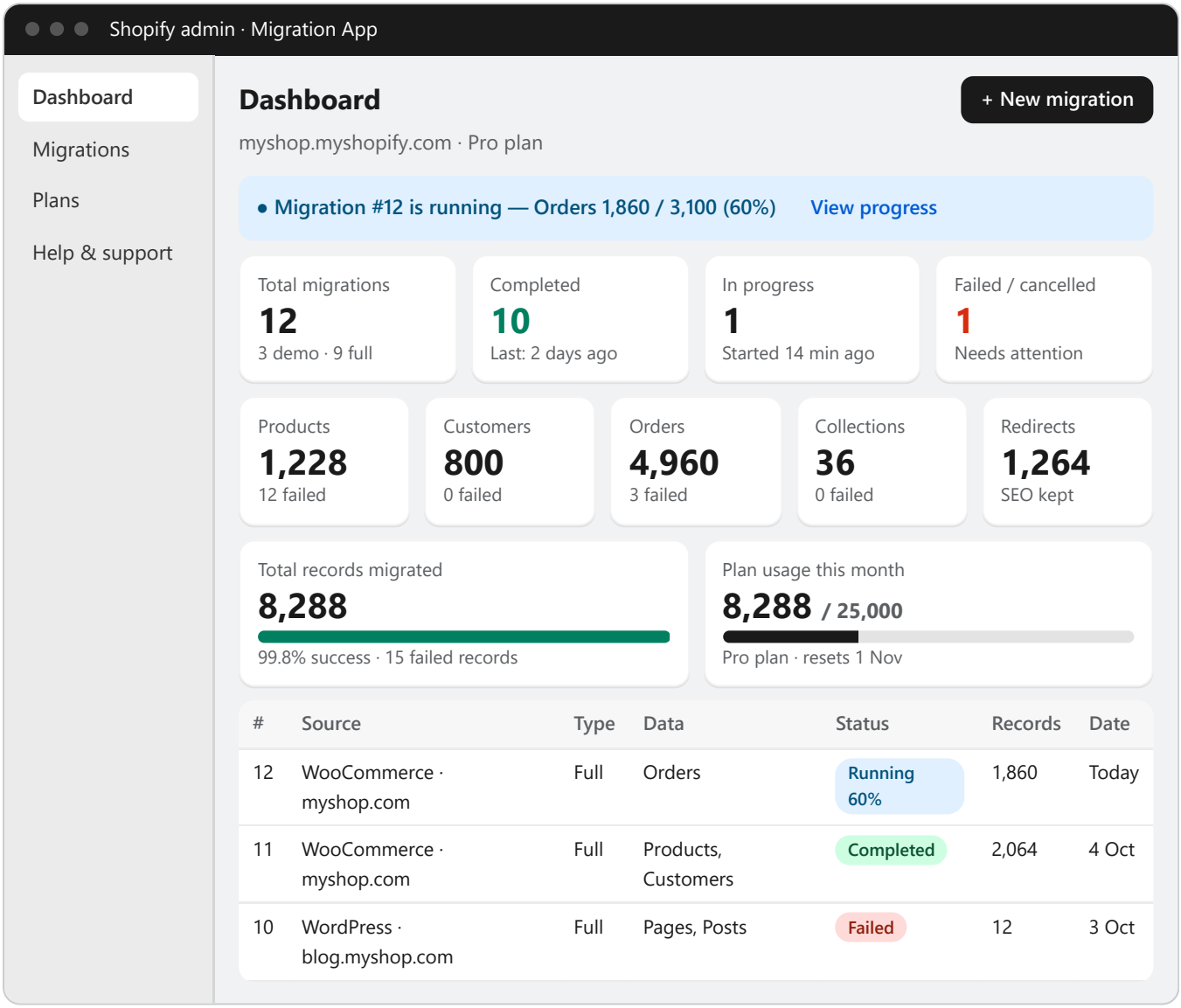
Right after install the app opens here until the steps are done or the merchant skips. Each step is ticked automatically from the database — nothing is ticked by hand.



Step	Ticked automatically when
1. Connect your old store	`source_connections` has a row with status `connected`
2. Run a free demo migration	`migrations` has a `demo` row with status `completed`
3. Choose a plan	`user_memberships` has an active row

Finishing or skipping stamps `stores.onboarded_at`; from then on the app opens on the Dashboard.

3. Dashboard

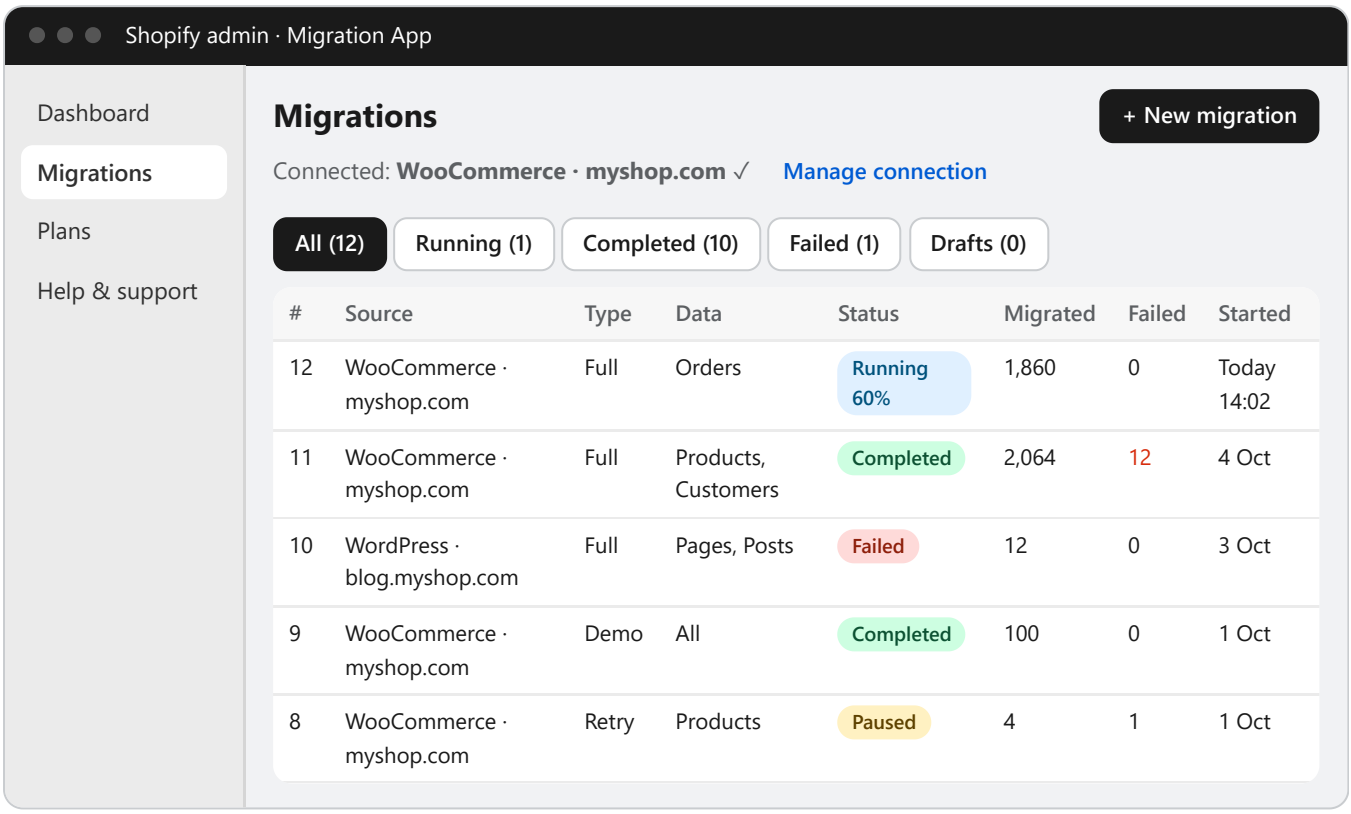


Where each card's number comes from

Card	Read from
Total / Completed / In progress / Failed	`migrations` — counted by `status` and `type`
Products, Customers, Orders, Collections, Redirects	`migration_steps` — `succeeded` and `failed` added up per `entity`
Total records migrated	Sum of all `migration_steps.succeeded` (demo runs excluded)
Plan usage	This month's succeeded records vs. `plans.max_records` of the store's plan
Running banner	The `migrations` row with status `running` and its current step
Recent migrations	Last 5 `migrations` rows with their `source_connections`

4. Migrations list

The **Migrations** menu. Every migration this store has run, newest first, filterable by status. Clicking a row opens its progress or results.

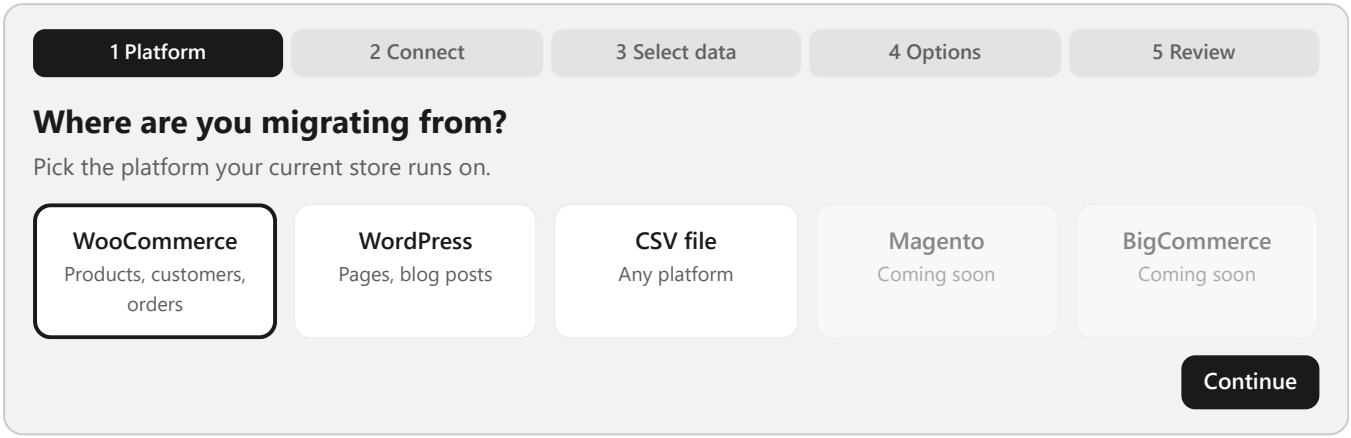


Reads `migrations` joined to `source_connections` and the totals of `migration_steps`. "New migration" creates a `migrations` row with status `draft` and opens the wizard.

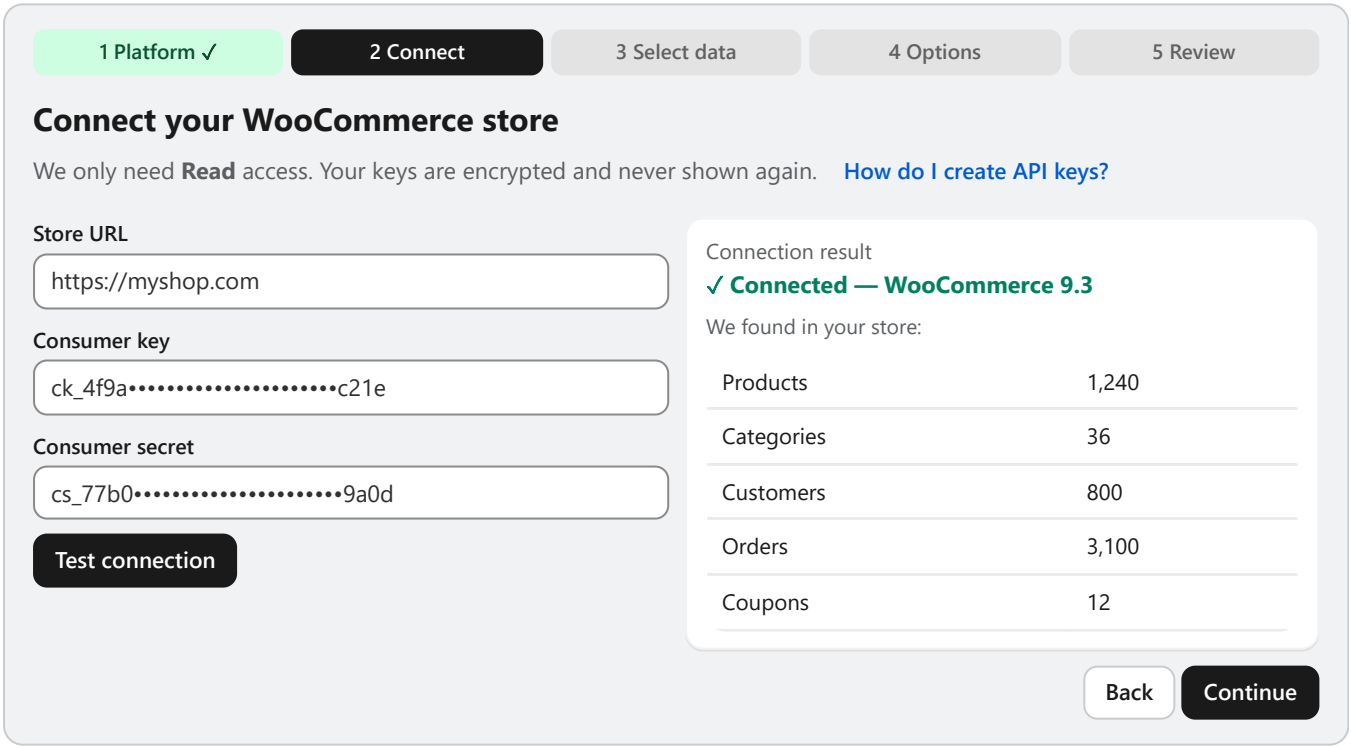
5. New migration — 5-step wizard

Each step is saved as the merchant goes (`migrations.wizard_step`), so leaving and returning continues where they stopped.

Step 1 — Choose platform

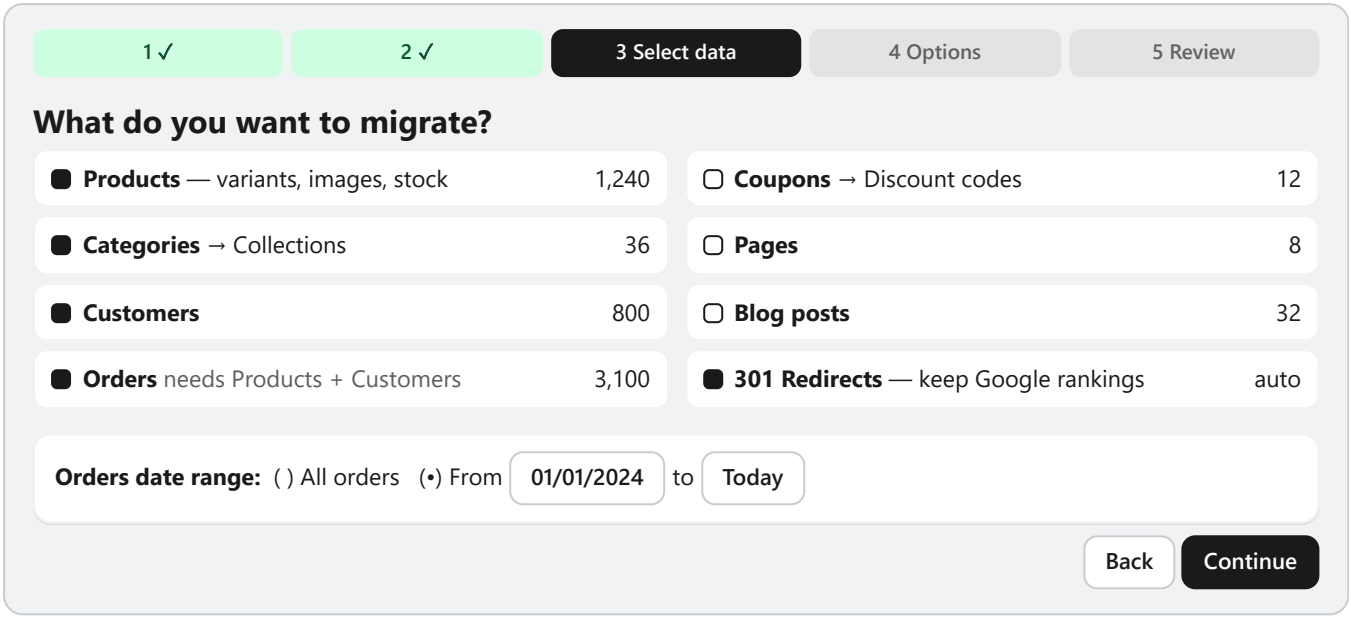


Step 2 — Connect: enter the keys



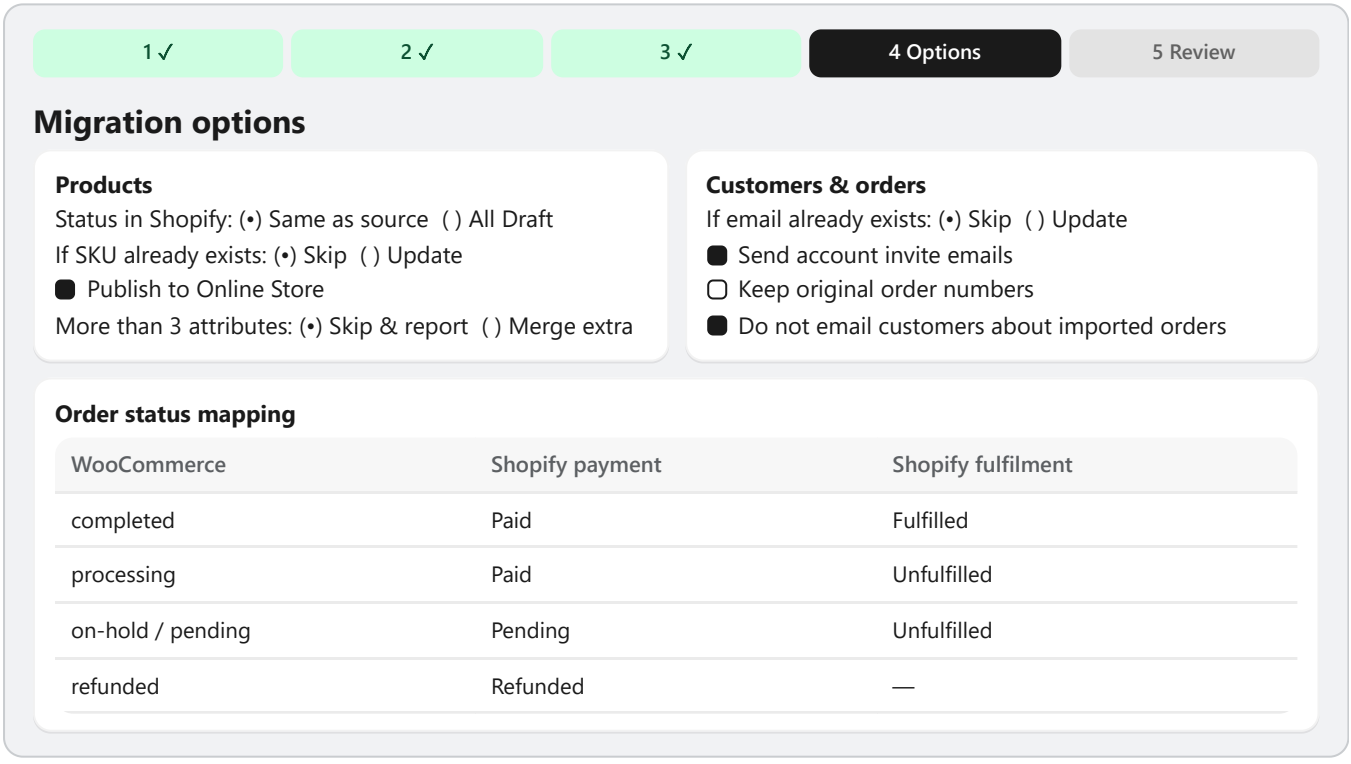
Saved to `source_connections`: URL, encrypted keys, masked hint, status, WooCommerce version and the counts (`scan_counts`).

Step 3 — Select what to migrate



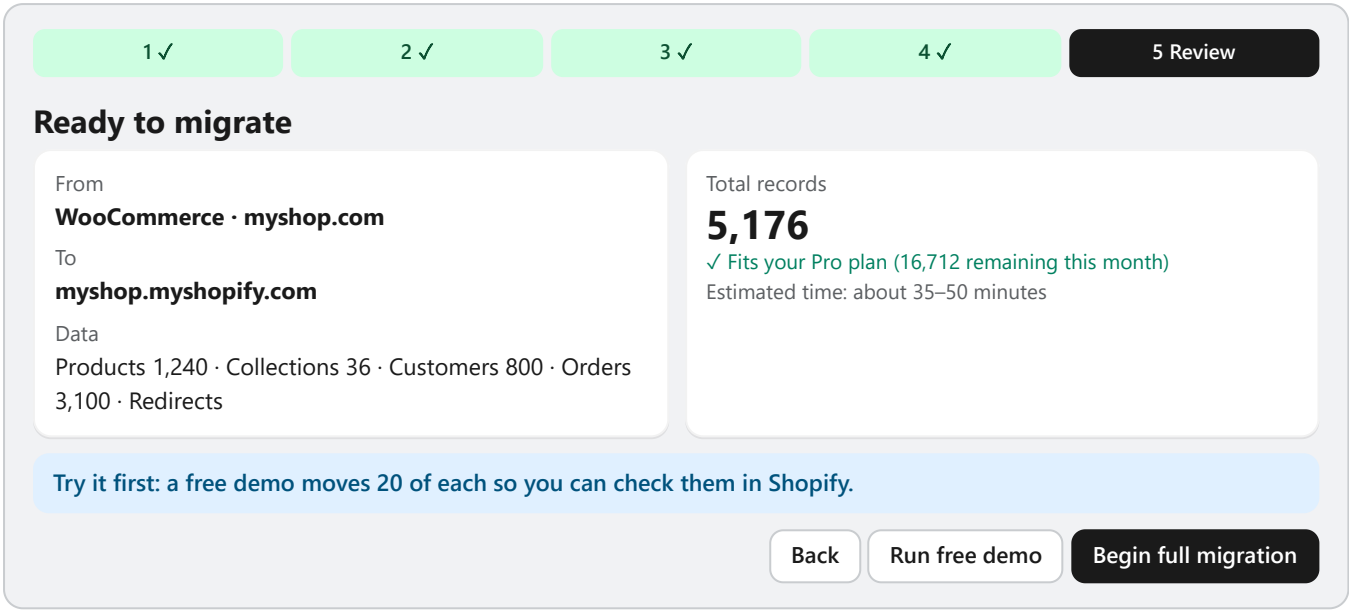
Saved to `migrations.entities`, e.g. `["collections","products","customers","orders","redirects"]`.

Step 4 — Options



Saved to `migrations.options` as JSON.

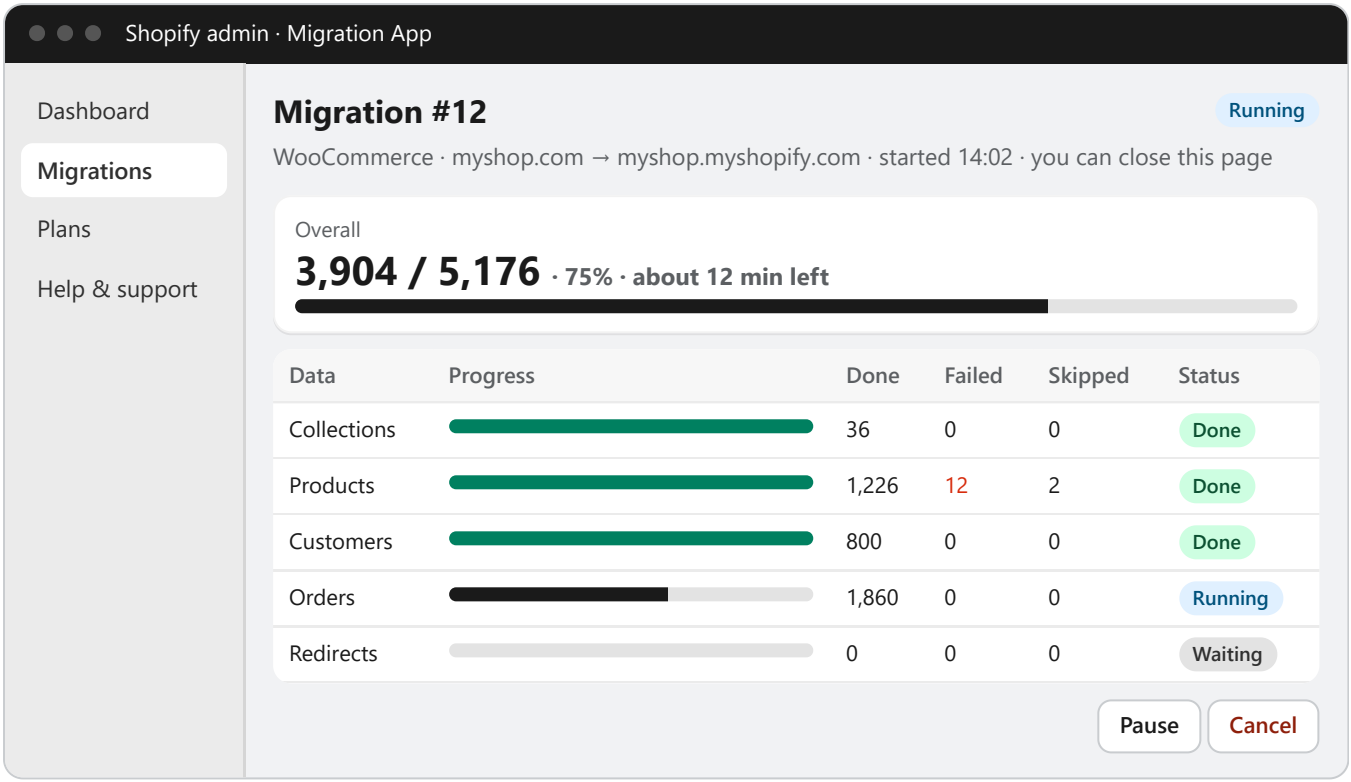
Step 5 — Review and begin



Pressing **Begin**: the plan limit is checked against `plans.max_records`, `migrations.status` becomes `queued`, one `migration_steps` row is created per selected data type, and the progress screen opens. If the plan is too small, the button becomes **Upgrade plan**.

6. Progress screen

Updates itself every few seconds. The merchant can close it — the migration keeps running on the server.



On screen	Read from
Each row and bar	One `migration_steps` row: `total`, `succeeded`, `failed`, `skipped`, `status`
Overall	Totals of those rows; time left from `migrations.started_at` and the speed so far
Pause / Cancel	Change `migrations.status`. Pause stops after the current page; Resume continues from `source_cursor`.

7. Results screen

Migration #11

Completed

✓ Finished in 42 minutes — 2,064 records migrated, 12 need attention.

Migrated	Failed	Skipped (already existed)	Duration
2,064	12	2	42 min

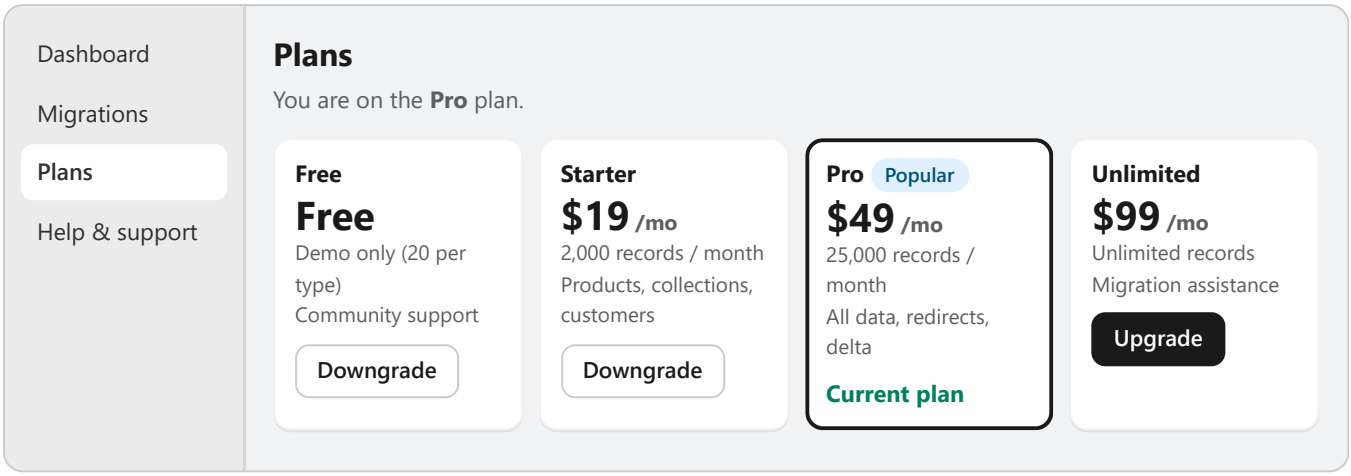
Data	Old ID	Name	Reason
Product	4512	Linen Shirt	Has 4 attributes; Shopify allows 3 options
Product	4630	Gift Box	Image URL returned 404
Product	4701	Mug Set	Grouped product — not supported

Download error CSV | Retry failed (12) | Migrate recent data | View in Shopify

Button	What it does in the database
Error list / Download error CSV	Reads `migration_errors` for this migration
Retry failed	New `migrations` row, type `retry`, `parent_migration_id` = this one; re-runs only unresolved `migration_errors`
Migrate recent data	New `migrations` row, type `delta`; `id_map` makes it bring only new or changed records
Delete demo data (demo runs only)	Deletes from Shopify every record in `id_map` with this `migration_id`

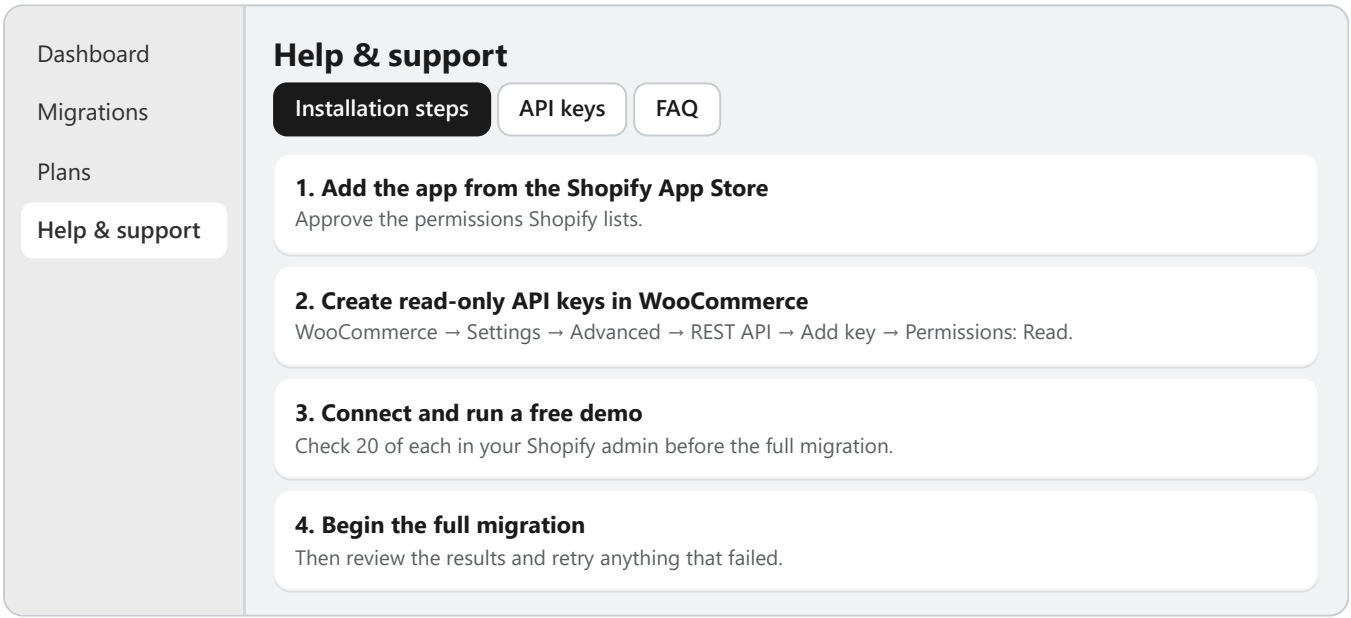
8. Plans and Help & support

Plans



Cards come from `plans`; the current plan from `user_memberships`; Shopify charges are recorded in `membership_payments`. Prices shown are examples.

Help & support



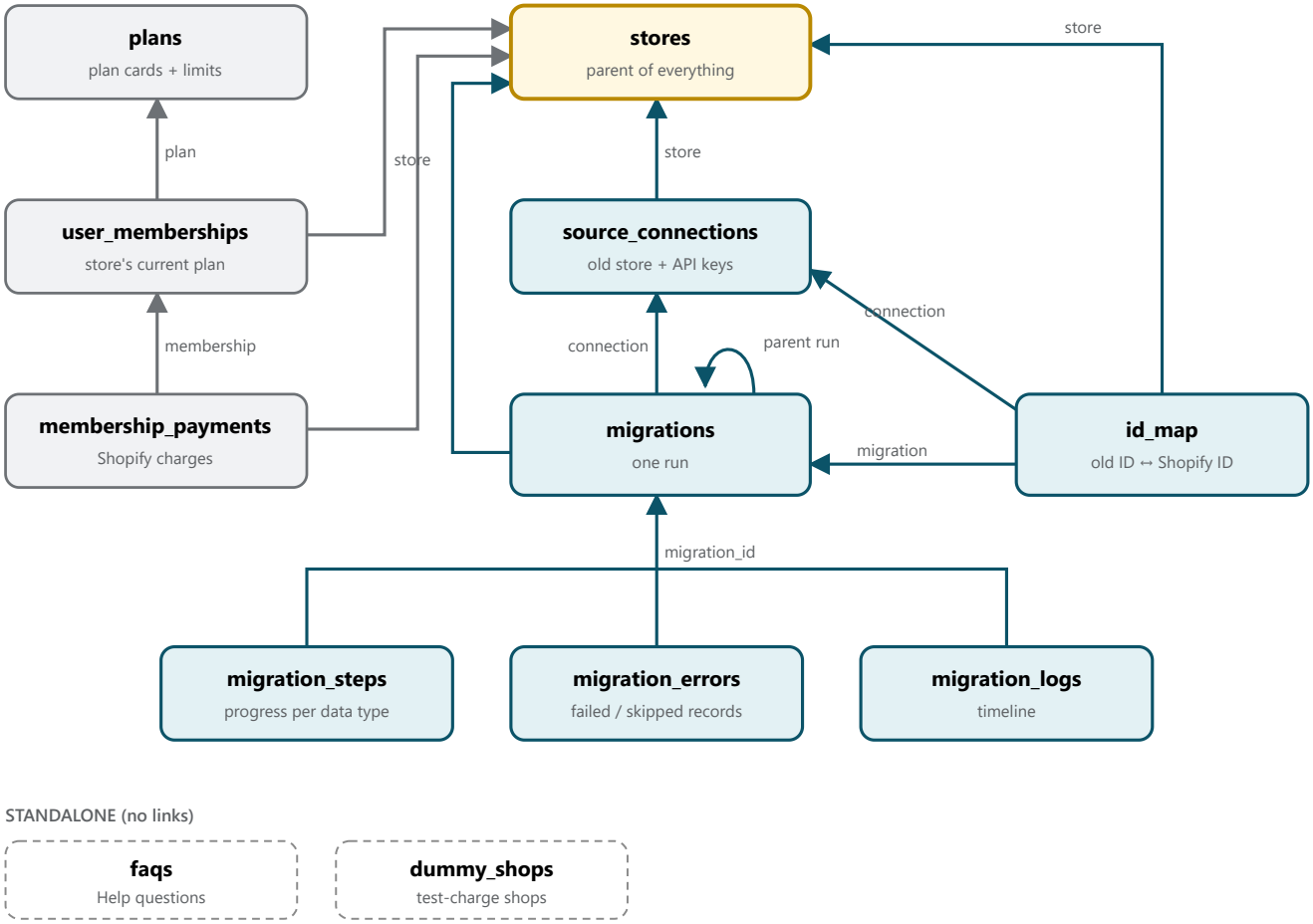
FAQ questions come from the `faqs` table, so they can be edited without a code change.

9. Database overview and relationship diagram

Database name **`migration_tool`**. The app creates it and all tables automatically on start (`config/migrate.js`); `sql/schema.sql` has the same tables for manual setup.

#	Table	Group	Stores
1	`stores`	Install	Every Shopify store that installed the app, with its encrypted Shopify token
2	`plans`	Billing	Plan cards and their limits
3	`user_memberships`	Billing	Which plan each store is on
4	`membership_payments`	Billing	Shopify subscription charges
5	`dummy_shops`	Billing	Shops that get free test charges
6	`faqs`	Help	Help & support questions and answers
7	`source_connections`	Migration	The merchant's old store: URL, encrypted API keys, what was found
8	`migrations`	Migration	One migration run: type, status, chosen data and options
9	`migration_steps`	Migration	Progress of each data type in a run (the progress bars)
10	`id_map`	Migration	Old record ID ↔ new Shopify ID for everything migrated
11	`migration_errors`	Migration	Records that failed or were skipped, with the reason
12	`migration_logs`	Migration	Timeline of a run, for support

Relationship diagram



Arrows point from a table to the table it belongs to (child → parent). Migration tables Install & billing tables stores

> **In one sentence:** a **store** connects one or more **old stores** (`source_connections`); each connection runs **migrations**; each migration has a **step** per data type plus **errors** and **logs**; and **id_map** remembers every old record and the Shopify record it became.

10. Table by table

Key: PK primary key · FK link to another table · UK must be unique

1. stores

Install

One row per Shopify store that installed the app. The parent of every other linked table.

Column	Type	Key	What it holds
id	INT	PK	
shop_domain	VARCHAR(255)	UK	myshop.myshopify.com
store_name, email, currency	VARCHAR		Shop details from Shopify
access_token, refresh_token	TEXT		Shopify login — **encrypted**
access_token_expires_at, refresh_token_expires_at	DATETIME		When the tokens expire (refreshed automatically)
api_version	VARCHAR(16)		Shopify API version used
is_active, installed_at, uninstalled_at	TINYINT, DATETIME		Install state
onboarded_at	TIMESTAMP		NULL = app still opens on Setup
created_at, updated_at	TIMESTAMP		

2. plans

Billing

The plan cards and what each plan allows.

Column	Type	Key	What it holds
id	INT	PK	
name	VARCHAR(255)	UK	Free, Starter, Pro…
price, days	DOUBLE, INT		USD per 30 days
is_popular, is_active	TINYINT		"Popular" badge; on sale or not
plan_content	LONGTEXT (JSON)		Lines shown on the card
max_records	INT		Records per month · NULL = unlimited
max_migrations	INT		Full migrations per month · NULL = unlimited
allowed_entities	JSON		Data types allowed, e.g. ["products","customers"] · NULL = all
created_at, updated_at	DATETIME		

3. user_memberships

Billing

Which plan each store is on, and its plan history.

Column	Type	Key	What it holds
id	INT	PK	

user_id	INT	FK	→ `stores.id`
membership_id	INT	FK	→ `plans.id`
status	TINYINT		1 = current plan · 0 = old, or paid plan waiting for approval
created_at, updated_at	DATETIME		

4. membership_payments

Billing

One row per Shopify subscription charge.

Column	Type	Key	What it holds
id	INT	PK	
membership_id	INT	FK	→ `user_memberships.id`
user_id	INT	FK	→ `stores.id`
charge_id, api_client_id	BIGINT		Shopify's charge ID
status	INT		1 = waiting for approval · 2 = active · 0 = declined
date_add, date_update	DATETIME		

5. dummy_shops · 6. faqs

Standalone

Table / column	Type	Key	What it holds
dummy_shops.id	INT	PK	
shop_name, status	VARCHAR, TINYINT		Shops listed with status 1 get Shopify test charges (no money)
faqs.id	INT	PK	
question, answer	VARCHAR(512), TEXT		One Help question
position, is_active	INT, TINYINT		Display order; shown or hidden
created_at, updated_at	TIMESTAMP		

7. source_connections

Migration

The merchant's old store — written by wizard Step 2.

Column	Type	Key	What it holds
id	INT	PK	
store_id	INT	FK	→ `stores.id`
platform	ENUM		woocommerce · wordpress · csv · magento · bigcommerce
base_url	VARCHAR(255)		https://myshop.com
credentials	TEXT		API keys as JSON — **encrypted**
credentials_hint	VARCHAR(64)		Masked value shown on screen: ck_4f9a…c21e

status, last_error	ENUM, VARCHAR		connected · failed · disconnected, and why
source_version	VARCHAR(64)		WooCommerce 9.3
scan_counts	JSON		{"products":1240,"customers":800,"orders":3100}
last_tested_at, created_at, updated_at	DATETIME / TIMESTAMP		

8. migrations

Migration

One migration run — created by "New migration", filled by the wizard.

Column	Type	Key	What it holds
id	INT	PK	Shown as "Migration #12"
store_id	INT	FK	→ `stores.id`
source_connection_id	INT	FK	→ `source_connections.id` — migrating from
type	ENUM		demo · full · delta · retry
status	ENUM		draft · queued · running · paused · completed · failed · cancelled
wizard_step	TINYINT		1–5 while a draft
entities	JSON		Step 3 choices: ["products","customers","orders"]
options	JSON		Step 4 choices
total_records	INT		Records to migrate (plan check, % bar)
parent_migration_id	INT	FK	→ `migrations.id` — the run a retry/delta came from
worker_id, locked_at	VARCHAR, DATETIME		Stops two workers running the same migration
last_error	VARCHAR(512)		Why it failed
started_at, finished_at, created_at, updated_at	DATETIME / TIMESTAMP		

9. migration_steps

Migration

Progress of one data type within a run — the progress bars and dashboard cards.

Column	Type	Key	What it holds
id	INT	PK	
migration_id	INT	FK UK	→ `migrations.id` · unique with entity
entity	VARCHAR(32)	UK	collections · products · customers · orders · coupons · pages · posts · redirects
position	TINYINT		Run order (collections first … redirects last)
status	ENUM		pending · running · done · failed · skipped
total, processed, succeeded, failed, skipped	INT		Live counters

source_cursor	VARCHAR(255)		Where to continue in the old store after pause/restart
started_at, finished_at	DATETIME		

10. id_map

Migration

Every old record and the Shopify record it became. The most important migration table.

Column	Type	Key	What it holds
id	BIGINT	PK	
store_id	INT	FK	→ `stores.id`
source_connection_id	INT	FK UK	→ `source_connections.id` · unique with entity + source_id
entity, source_id	VARCHAR	UK	product + 4512 (ID in WooCommerce)
source_url	VARCHAR(512)		Old page address → source of the 301 redirect
shopify_gid	VARCHAR(128)		gid://shopify/Product/8123456789
shopify_handle	VARCHAR(255)		New page address → target of the redirect
source_hash	CHAR(40)		Fingerprint of the old record — has it changed since?
migration_id	INT	FK	→ `migrations.id` — the run that created it
created_at, updated_at	TIMESTAMP		

> Used for: **no duplicates** (one row per old record) · **linking** (an order finds its product's Shopify ID here) · **301 redirects** (source_url → shopify_handle) · **delta runs** (source_hash) · **delete demo data** (migration_id).

11. migration_errors

Migration

Records that failed or were skipped — the results screen's error list, Retry and CSV.

Column	Type	Key	What it holds
id	BIGINT	PK	
migration_id	INT	FK UK	→ `migrations.id` · unique with entity + source_id
entity, source_id, source_name	VARCHAR	UK	product · 4512 · "Linen Shirt"
kind	ENUM		failed · skipped
error	TEXT		The reason, in plain words
attempts, resolved	TINYINT		Retry count; 1 once a retry succeeds
created_at, updated_at	TIMESTAMP		

12. migration_logs

Migration

Timeline of a run, for support.

Column	Type	Key	What it holds

id	BIGINT	PK	
migration_id	INT	FK	→ `migrations.id`
level	ENUM		info · warn · error
message	VARCHAR(1024)		"Products step started", "Old store returned 503, retrying"
created_at	TIMESTAMP		Logs older than 90 days are removed

11. Relationships and delete rules

Every link

From (child)	To (parent)	One → many	If the parent is deleted
`user_memberships.user_id`	`stores.id`	1 store → many	Deleted too
`user_memberships.membership_id`	`plans.id`	1 plan → many	**Blocked** — a plan in use cannot be deleted
`membership_payments.membership_id`	`user_memberships.id`	1 → many	Deleted too
`membership_payments.user_id`	`stores.id`	1 store → many	Deleted too
`source_connections.store_id`	`stores.id`	1 store → many	Deleted too (keys removed)
`migrations.store_id`	`stores.id`	1 store → many	Deleted too
`migrations.source_connection_id`	`source_connections.id`	1 connection → many	Deleted too
`migrations.parent_migration_id`	`migrations.id`	1 run → its retries	Link cleared (set to NULL), retry run kept
`migration_steps.migration_id`	`migrations.id`	1 run → 1 per data type	Deleted too
`migration_errors.migration_id`	`migrations.id`	1 run → many	Deleted too
`migration_logs.migration_id`	`migrations.id`	1 run → many	Deleted too
`id_map.store_id`	`stores.id`	1 store → many	Deleted too
`id_map.source_connection_id`	`source_connections.id`	1 connection → many	Deleted too
`id_map.migration_id`	`migrations.id`	1 run → many	Link cleared (set to NULL), mapping kept

What a delete removes

When this is deleted…	…this goes with it
A store (Shopify's shop/redact after uninstall)	Everything it owns: memberships, payments, connections and keys, migrations, steps, errors, logs, id_map
A connection (merchant presses Disconnect)	Its keys, its migrations with their steps/errors/logs, and its id_map rows
A migration	Its steps, errors and logs. id_map rows **stay**, so a later run still knows what was migrated
A plan	Refused while any store is on it

12. Example: what each step writes to the database

#	Merchant does	Database
1	Installs the app	**stores** +1 row (token encrypted)
2	Picks the Free plan	**user_memberships** +1, **membership_payments** +1
3	Wizard Step 2: enters WooCommerce keys, tests	**source_connections** +1 (keys encrypted, scan_counts filled)
4	Opens "New migration"	**migrations** +1, status `draft`
5	Steps 3–4: ticks Products, Customers, Orders; sets options	**migrations**.entities and .options updated
6	Step 5: presses Begin	**migrations**.status = `queued`; **migration_steps** +3 (products, customers, orders)
7	(background) a product is created in Shopify	**id_map** +1; that step's `succeeded` +1
8	(background) a product fails	**migration_errors** +1; that step's `failed` +1
9	(background) an order is created	Its products and customer found in **id_map**; **id_map** +1 for the order
10	(background) progress messages	**migration_logs** rows
11	All steps done	**migrations**.status = `completed`, finished_at set
12	Opens the Dashboard	Cards add up **migrations** and **migration_steps**
13	Presses Retry failed	**migrations** +1 (type retry, parent_migration_id set); fixed errors get resolved = 1